Conexão entre Frontend e Banco de Dados

Aula 20 · Projeto de Sistemas · Prof. Moisés Sousa · IDP

Como aplicações web acessam dados · Fluxo completo da informação · Exemplo prático com PHP + MySQL

Objetivos da aula

- Entender como frontend e banco de dados se comunicam.
- Compreender o papel do backend.
- Consumir dados com JavaScript (fetch + JSON).
- Construir uma aplicação simples integrada com PHP + MySQL.
- Conhecer os principais riscos de segurança e como evitá-los.

Problema central: o HTML acessa o banco?

NÃO

- Segurança, arquitetura e acesso ao servidor são os motivos.
- Vamos ver cada um deles nos próximos slides.

Por que o frontend não acessa o banco diretamente?

Segurança

O JS roda no navegador do usuário: qualquer senha no código fica visível (F12, "Ver código-fonte").

Protocolo

O navegador fala HTTP. O MySQL usa protocolo próprio (porta 3306).

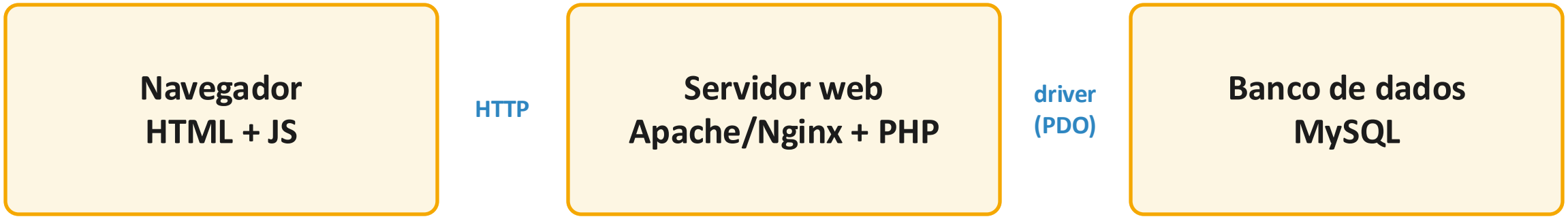
Exposição

O banco deve ser acessível só pelo servidor da aplicação, nunca pela internet.

Confiança

O frontend nunca é confiável: regras e validações precisam existir também no servidor.

Arquitetura da aplicação em camadas



- Frontend → Backend → Banco de Dados
- Separação de responsabilidades (apresentação, lógica, dados): base do padrão MVC

Papéis de cada camada

Frontend

Interface e interação.

NÃO guarda segredo.
NÃO decide regra de negócio.

Backend

Regras, validação e acesso ao banco.

NÃO mistura HTML com SQL.

Banco de dados

Armazenamento e consultas.

Garante integridade: PK, FK, constraints.

Tecnologias envolvidas

Tecnologia	Papel	Camada
HTML	Estrutura da página	Frontend
JavaScript	Interação e requisições (fetch)	Frontend
PHP	Regras e acesso ao banco	Backend
MySQL	Armazenamento e consultas	Banco

HTTP: o idioma da web

- Modelo cliente-servidor: o cliente pede, o servidor responde.
- HTTP é stateless: cada requisição é independente (por isso existem sessões e tokens).
- Uma API é o contrato entre frontend e backend: URL, método e formato da resposta.
- Assíncrono: o JS não trava a página enquanto espera a resposta do servidor.

Métodos HTTP × CRUD × SQL

Método HTTP	Ação	Comando SQL
GET	Buscar / ler	SELECT
POST	Enviar / criar	INSERT
PUT	Atualizar	UPDATE
DELETE	Remover	DELETE

Hoje: foco no GET (SELECT). Na próxima aula: POST, PUT e DELETE (CRUD completo).

Códigos de status HTTP

Código	Significado
200 OK	Requisição bem-sucedida
201 Created	Registro criado (POST)
400 Bad Request	Dados inválidos enviados pelo cliente
404 Not Found	URL ou recurso não encontrado
500 Internal Server Error	Erro no servidor (ex.: falha na conexão com o banco)

JSON: o formato de troca de dados

- Texto estruturado, leve e legível, suporte nativo em JS e PHP
- Substituiu o XML na maioria das APIs web
- PHP: json_encode() · JS: response.json()

```
[
   { "nome": "Ana",   "nota": 8.5 },
   { "nome": "Bruno", "nota": 5.0 }
]
```

Fluxo de comunicação completo

1. Usuário acessa a página

2. JS faz fetch() (assíncrono)

3. Servidor recebe e valida

4. PHP executa SQL (prepared statement)

5. Resposta volta em JSON

6. JS atualiza o DOM sem recarregar

Banco → PHP → JS → HTML

Ambiente de desenvolvimento

- Programas importantes: Docker Compose com php-apache + mysql + phpmyadmin (mesmo ambiente para todos)
- Cliente do banco: phpMyAdmin, DBeaver ou MySQL Workbench
- Editor: VS Code · Teste de API: navegador, Postman ou Insomnia

Passo 1: preparar o banco

```
CREATE DATABASE escola CHARACTER SET utf8mb4;
USE escola;

CREATE TABLE alunos (
  id   INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  nota DECIMAL(4,2) NOT NULL
);

INSERT INTO alunos (nome, nota) VALUES
  ('Ana', 8.5), ('Bruno', 5.0), ('Carla', 7.0);
```

Passo 2: backend - conexão com PDO

```php
<?php
// config.php  (fora da pasta pública!)
$host = 'localhost';
$db   = 'escola';
$user = 'app_escola';   // nunca root
$pass = 'senha_forte';

$pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8mb4", $user, $pass,
  [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]);
```

PDO é portável entre SGBDs e tem prepared statements nativos. Ciclo: conectar → preparar → executar → buscar → responder.

Passo 3: backend - consulta e resposta JSON

```php
<?php
// alunos.php
require __DIR__ . '/../config.php';

$stmt = $pdo->prepare('SELECT nome, nota FROM alunos');
$stmt->execute();

header('Content-Type: application/json; charset=utf-8');
echo json_encode($stmt->fetchAll(PDO::FETCH_ASSOC));
```

Teste: abra http://localhost/alunos.php no navegador e confira o JSON antes de fazer o frontend.

Passo 4: frontend - HTML base

```
<!DOCTYPE html>
<html lang="pt-BR">
<head><meta charset="utf-8"><title>Alunos</title></head>
<body>
  <h1>Lista de alunos</h1>
       <ul id="lista"></ul>
  <script src="app.js"></script>
</body>
</html>
```

Passo 5: frontend - fetch e DOM

```
const lista = document.getElementById('lista');

fetch('alunos.php')
  .then(r => r.json())
  .then(alunos => {
    alunos.forEach(a => {
      const li = document.createElement('li');
      const status = a.nota >= 7 ? 'Aprovado' : 'Reprovado';
      li.textContent = a.nome + ': ' + a.nota + ' - ' + status;
      lista.appendChild(li);
    });
  })
  .catch(e => console.error('Erro:', e));
```

Use textContent (e não innerHTML) ao inserir dados: evita XSS.

Erros comuns e sintomas

Sintoma	Causa provável
"Unexpected token < in JSON"	PHP imprimiu aviso/HTML antes do JSON
404 no fetch	Caminho do arquivo PHP errado
Erro de CORS no console	Frontend e backend em origens diferentes
Connection refused / Access denied	MySQL parado, porta ou credencial incorreta
Acentos quebrados	Charset diferente de utf8mb4
Tentar acessar o banco direto no JS	Falha de arquitetura: use sempre o backend

Segurança: SQL Injection

Vulnerável

```
$sql = "SELECT * FROM alunos
  WHERE id = " . $_GET['id'];
```

Seguro

```
$stmt = $pdo->prepare(
  'SELECT * FROM alunos WHERE id = ?');
$stmt->execute([$_GET['id']]);
```

- Entrada do usuário concatenada no SQL pode alterar a consulta
- Prepared statements separam o comando SQL dos dados

Segurança: outras boas práticas

- Não expor senha: credenciais em arquivo fora da pasta pública ou em variáveis de ambiente (.env), nunca no Git
- Menor privilégio: o usuário do banco da aplicação não deve ser root
- Validar dados no backend (tipo, tamanho, formato), mesmo que o frontend já valide
- Escapar a saída no DOM (textContent) para evitar XSS
- Usar HTTPS em produção e configurar CORS apenas para origens necessárias

Resumo e próxima aula

- Banco → PHP → JS → HTML: fluxo completo funcionando
- Arquitetura web em camadas, integração completa, base para sistemas reais

Próxima aula

Inserção de dados e CRUD completo (POST, PUT e DELETE)

Ferramentas e APIs para praticar

Categoria	Sugestões
Ambiente local	XAMPP, Laragon, Docker Compose (php-apache + mysql + phpmyadmin)
Cliente de banco	phpMyAdmin, DBeaver, MySQL Workbench
Teste de API	Postman, Insomnia, Bruno, DevTools (Network)
Backend alternativo	PHP (Slim, Laravel), Python (Flask, FastAPI), Node.js (Express)
APIs públicas (JSON)	JSONPlaceholder, ViaCEP, IBGE Localidades, Open-Meteo
Boas práticas	Git/GitHub + .gitignore, phpdotenv, OWASP Top 10

Exercício

- Criar a tabela alunos e inserir ao menos 5 registros
- Listar os alunos na página com JavaScript (fetch)
- Mostrar a nota de cada aluno
- Exibir o status: Aprovado (nota ≥ 7) ou Reprovado
- Desafio: faça o cálculo do status no backend (PHP) e compare: onde a regra de negócio deve ficar?